

Software Architecture
(T630019402)

Lecture #2, February 17, 2026

Jukka Ruohonen

Before we begin, any questions about practicalities?

Case #1 Presentations

- ▶ Consider Hrvoje's (2024) diagram about Linux
 - Online: <https://zenodo.org/records/14179366>
 - On *itslearning* as case-1.pdf
- ▶ What kind of an architecture it is?
- ▶ What components and connectors there are?
- ▶ Which components and connectors do you know?
- ▶ If you start from the applications on top (the black rectangle), can you explain how a package traverses through the components and connectors to a network interface?
- ▶ What other input and output connectors there are?
- ▶ Anything other interesting to comment and share?

Today's Lecture

- ▶ Today's lecture presents a high-level overview of essential **quality attributes** (QAs) against which architectures should be designed, keeping tensions in mind and making balancing choices between attributes whenever necessary
 - The attributes, tensions, and balancing are addressed in more detail throughout the subsequent lectures
- ▶ In addition, QAs are elaborated in terms of **requirements engineering** and the derivation of QAs in particular
 - Therefore, also a few points about **agile development**
- ▶ A remark: henceforth, the textbooks enumerated in the first lecture are referenced (cf., e.g., Bass et al. 2021) without them appearing in per-lecture additional bibliographies

Software Architecture and Agile Development (1/5)

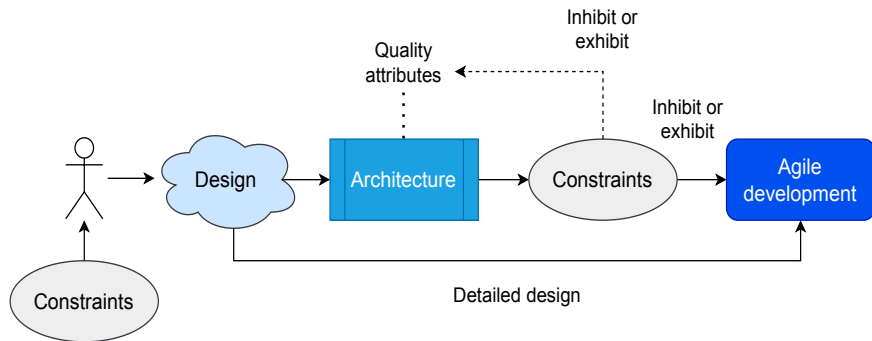


Figure 1: Does It Inhibit or Exhibit Agile Development?

Software Architecture and Agile Development (2/5)

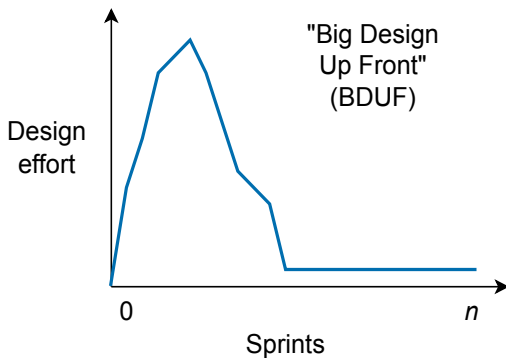


Figure 2: Intuitively, Software Architecture Would Indeed Seem to Inhibit Agile Development, as Also Many Popular Concepts Hint¹

¹ https://en.wikipedia.org/wiki/Big_design_up_front

Software Architecture and Agile Development (3/5)

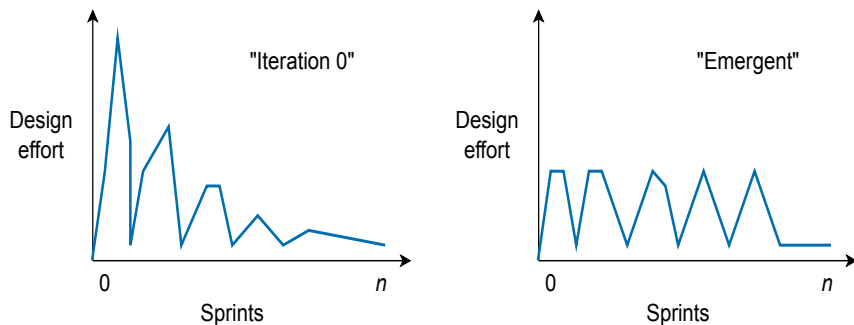


Figure 3: However, After a Second Thought, It Is Not Presumably True or at Least Not Straightforward (adopted from Bass et al. 2021, Fig. 24.1)

Software Architecture and Agile Development (4/5)

- ▶ Agile development has been the dominant development model for over twenty years or so, and great architectures have been designed and implemented in that time!
 - Also: while an extreme variant of “*architecture will emerge as we code*” might work for small projects, do you think it would work for the manufacturing automation example?
- ▶ Of the two previous models, “**Iteration 0**” seems generally appealing (Yang et al. 2016), and also because architectural design choices are still the hardest-to-change ones
 - ⇒ Perhaps start either from a simpler “*big picture design*” or a simple “*skeleton design*” and adjust once iterations are completed and new features have been implemented

Software Architecture and Agile Development (5/5)

- ▶ The systematic literature review of Yang et al. (2016) provide many further tips to improve the alignment between architecture and agile development – among these are:
 1. **Stakeholders** – including through their non-functional requirements from which quality controls are derived – should engage in decisions about any **freezes** made to an architecture
 2. Also **track architectural issues** in a backlog
 3. Keep architectural **documentation up to date**, and also **document iteratively and incrementally** as you go
 4. **Use tools** (from photographed blackboard design sketches to ticketing systems and at least occasionally formal tools)
- ▶ While they list many more good practices, many of them reflect what was said previously about job advertisements

Quality Attributes

- ▶ Bass et al. (2021) define a quality attribute as *“a measurable or testable property of a system that is used to indicate how well the system satisfies the needs of its stakeholders beyond the basic function of the system”*
 - Thus, a QA is related to stakeholders via their needs
 - Yet, a “need $\neq$ basic function” of a system
- ▶ Notice also the phrasing “measurable or testable”
 - Often easier said than done; how to measure security, say?
- ▶ Note also that QAs are not only about source code; many of them require understanding a system’s **runtime** behavior

Quality Attributes Considered in the Course

- ▶ There are many—almost infinite—quality attributes
- ▶ The course covers some of traditional, “core QAs”, including those related to performance, scalability, availability, safety and security, and maintainability
 - In addition and depending on time, additional QAs, such as energy efficiency, are addressed to some extent
- ▶ Most of the QAs are difficult to rigorously define due to various issues, some of which will be soon discussed
- ▶ However, it is possible to refine them in a sense that $QA_i \mapsto \{SubQA_{i1}, SubQA_{i2}, \dots, SubQA_{in-1}, SubQA_{in}\}$ and $SubQA_{ij} \mapsto \{Metric_{ij1}, Metric_{ij2}, \dots, Metric_{ijm}\}$

Quality Attributes Considered in a Standard (1/3)

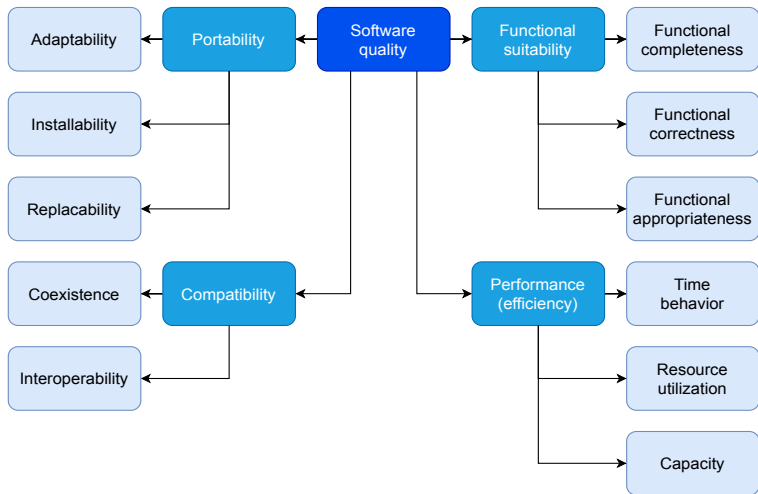


Figure 4: QAs in ISO/IEC 25010 (from Bass et al. 2021, Fig. 14.1)

Quality Attributes Considered in a Standard (2/3)

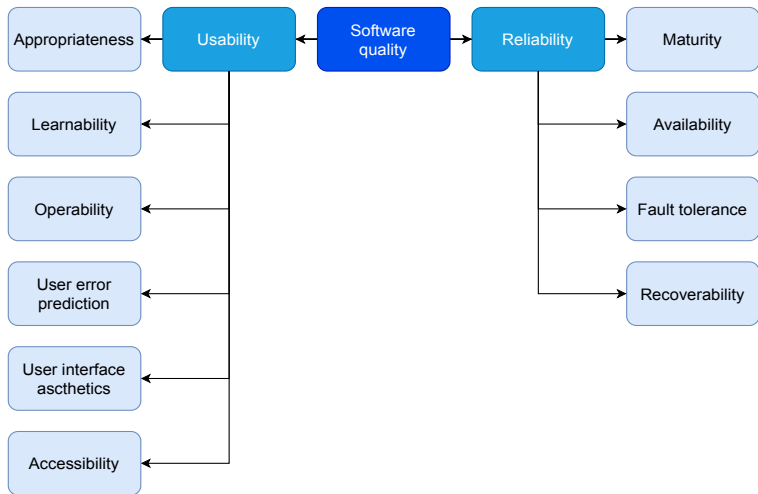


Figure 5: QAs in ISO/IEC 25010 (from Bass et al. 2021, Fig. 14.1)

Quality Attributes Considered in a Standard (3/3)

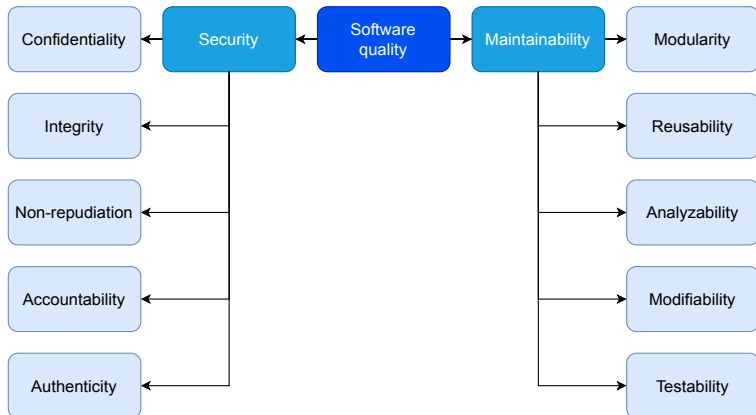


Figure 6: QAs in ISO/IEC 25010 (from Bass et al. 2021, Fig. 14.1)

The Antonyms of Quality Attributes

- ▶ There are many antonyms for quality attributes
- ▶ Fairbanks (2010) alongside many others use a term **big ball of mud** for a system that has (a) no clear connectors between elements; (b) not even a single QA promoted; (c) and no considerations about any constraints or downsides
 - Thus: even when all functional requirements have been met, a system may be a nightmare in terms of software quality
 - Concepts such as **technical debt** convey a message that many systems are redesigned and re-implemented but seldom due to their functionalities as such

Quality Attributes as Non-Functional Requirements (1/4)

- ▶ Another way to look at quality attributes is to frame them as **non-functional** requirements for a system
- ▶ However, the **distinction** between functional and non-functional requirements is fuzzy, confusing, and debated
 - A traditional way to frame functional requirements would be to link them to utility or value (including concrete business value in the private sector) of stakeholders (incl. customers)
 - Yet, problems still exist; Bass et al. (2021) give an example of designing a control engine; say, if we are designing an engine for a sports car whose “function” is to drive fast, can it really be designed without considering performance (QA) and measurements (e.g., acceleration and other timings) for it?

Quality Attributes as Non-Functional Requirements (2/4)

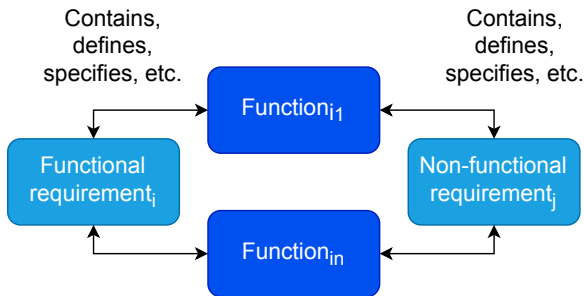


Figure 7: A Sketch for Approaching the Distinction

Quality Attributes as Non-Functional Requirements (3/4)

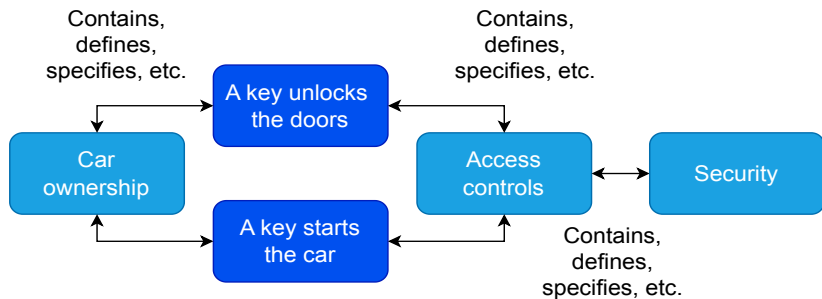


Figure 8: Another Sketch for Approaching the Distinction

Quality Attributes as Non-Functional Requirements (4/4)

- ▶ One way to deal with non-functional requirements (QAs) is to try to decompose them **toward functional requirements**
- ▶ A QA saying that “*a system should be secure*” is worthless
 - A security-QA saying that “*a system should comply with the PCI DSS standard*” is already much better (but not perfect)
- ▶ A QA saying that “*a system should scale*” is also worthless
 - However, a scalability-QA saying that “*a web application should be able to scale up to ten million users after about a year in operation*” is already much better (but not perfect)
- ▶ Yet, **overlapping concepts** still cause problems, including with respect to **domain knowledge** causing tensions too

Quality Attributes and Overlapping Concepts (1/2)

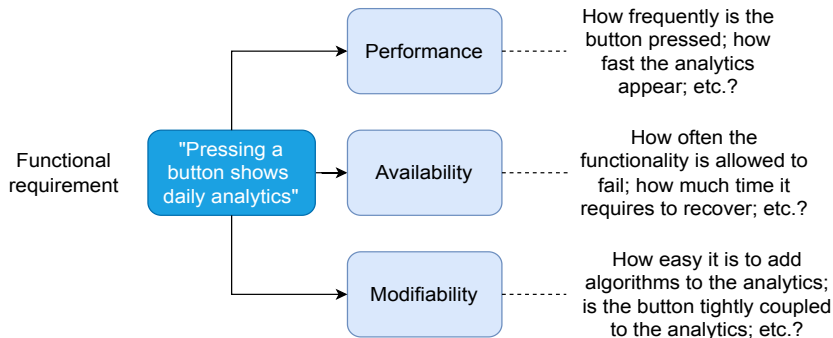


Figure 9: An Illustration of Overlapping Concepts

Quality Attributes and Overlapping Concepts (2/2)

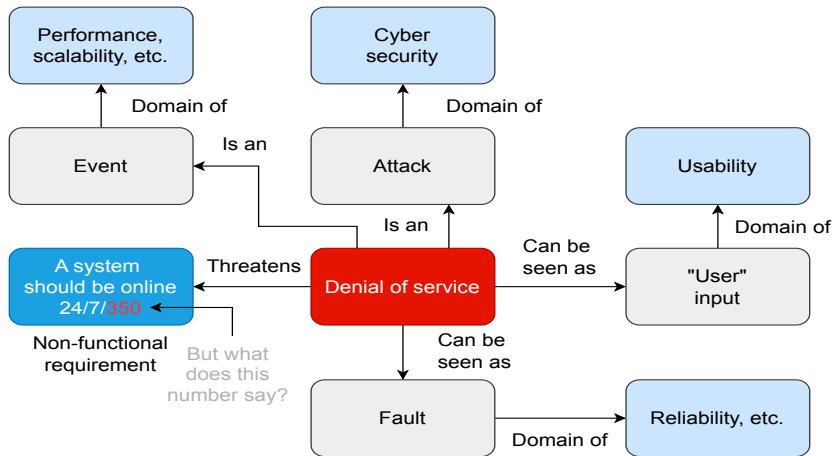


Figure 10: Another Take on Overlapping Concepts (Bass et al. 2021)

Quality Attributes via Scenarios (1/2)

- ▶ Whichever terminology you use, be **consistent**
- ▶ With a given terminology, a further useful way to approach QAs is through **scenarios** involving some **input and output**
 - In what follows, the terms **event** (input) and **response** (output) are used but without a preference over a vocabulary
- ▶ In general, QA scenarios align well with agile practices (cf. user stories, use cases, misuse cases, etc.)

Quality Attributes via Scenarios (2/2)

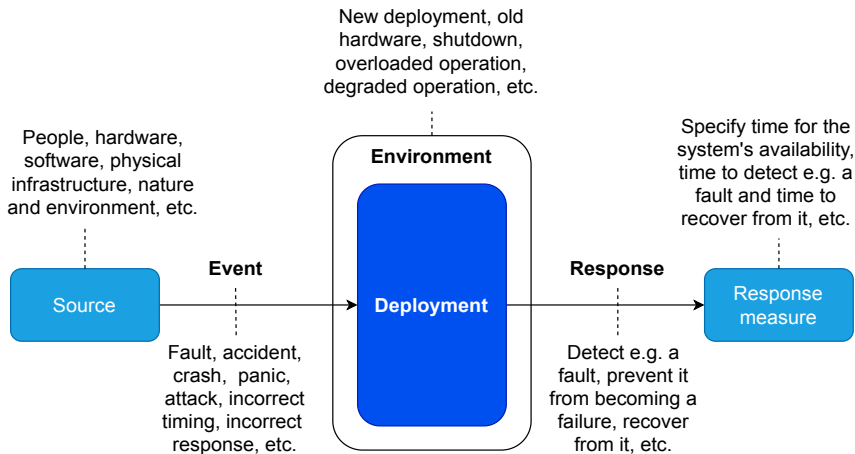
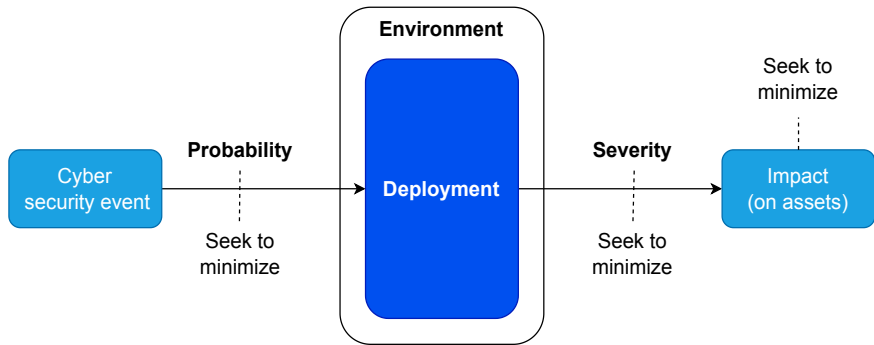


Figure 11: A QA Scenario Template (adopted from Bass et al. 2021)

Quality Attributes via Risks (1/3)

- ▶ Some QAs (such as **cyber security**) and application domains (such as **safety-critical systems**) are often perceived and modeled through **risks and risk analysis**
- ▶ The later lectures cover risk-based approaches in more detail; for the present purposes, it suffices to understand that in these domains a risk is generally defined as a probability of an event occurring and its impact upon some assets that are valuable to stakeholders (including software developers)
 - **Risk = Probability × Impact**; or sometimes also as
 - **Risk = Probability × Severity × Impact**; and so forth

Quality Attributes via Risks (2/3)



Quality Attributes via Risks (3/3)

- ▶ The risk-based approaches are useful also for **prioritizing** QAs, including responses and response measures taken (if any)
- ▶ For instance, consider that Event := *"Our small on-premise data center at the top of a mountain gets covered by a flood"*
 - Probability := Negligible (because it is on a mountaintop!)
 - Severity := High (maybe)
 - Impact := High (likely)

⇒ It is reasonable to bypass the scenario
- ▶ On the other hand, the earlier sports car example on p. 18 would likely lead to a conclusion about a "must"
- ▶ Once you have done prioritization via risk analysis or by other means, Fairbanks (2010, p. 150) suggest simple **rankings**, such as *"scalability > modifiability > security > usability"*

Cascading Risks and Impacts (1/4)

- ▶ Regarding fault-tolerance and cyber security, the concepts of **cascading risks** and **cascading impacts** are important
 - In other words, there is not just a risk but multiple risks and their impacts, which are (or may be) related to each other
 - **Conditional probability** is required
- ▶ While these make formal modeling challenging, it is still often useful to consider them for better understanding risk scenarios
 - Say, what happens if a major power failure crashes many servers, which, in turn, causes traffic to be routed to already overloaded backup servers, which, in turn, causes congestion control and throttling to set in?

Cascading Risks and Impacts (2/4)

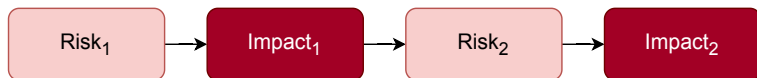


Figure 12: An Analytical Meaning of Cascading Risks and Impacts

Cascading Risks and Impacts (3/4)

While the path from risk₁ and impact₁ to risk₂ could be seen as a **cascade**, the realization of risk₃ through risk₂ and its impact₂ could already be seen as a full-blown **escalation**.

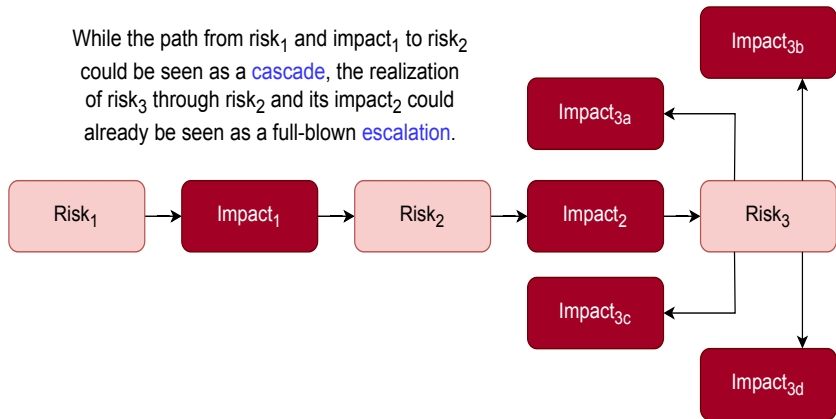


Figure 13: Cascading Further Toward an Escalation

Question

If we consider cyber security, what kind of a theoretical yet fundamental design principle we might use to counter cascading risks and impacts?

Cascading Risks and Impacts (4/4)

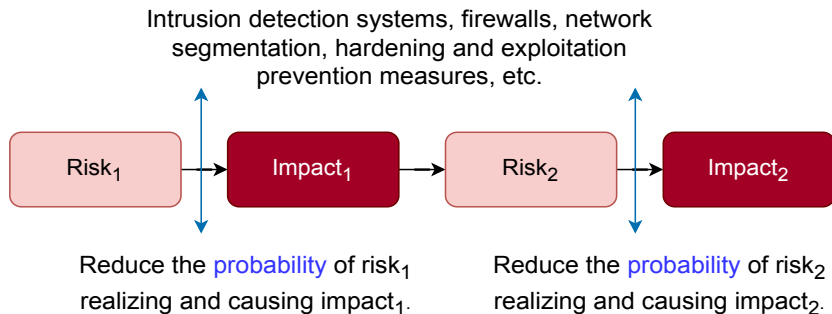


Figure 14: One Formulation for the Principle of **Defense in Depth**

Software Engineering and Risks More Generally (1/5)

- ▶ A risk-based approach is suitable also for software architectures and software engineering more generally

Table 1: Example Risks (adopted from Fairbanks 2010, p. 40)

Project management risks	Software architecture risks
"Lead developer hit by a bus"	"A system will not scale to a hundred thousand users"
"Stakeholder needs not understood for an architecture"	"The system is working now but we fear that if we touch anything it may fall apart"

Software Engineering and Risks More Generally (2/5)

- ▶ Fairbanks (2010, p. 48) argues that software architecture *“efforts should be commensurate with the risk of failure”*
 - Here, failure is most of all **project failure** too
- ▶ He continues to elaborate that with a risk analysis one should find the *“hot spots”* (with respect to probabilities and impacts) that *must* be designed and implemented
- ▶ If you identify a risk needing attention, you should also seek means to reduce the risk, and preferably with known tactics and patterns instead of rolling your own *ad hoc* solutions

Software Engineering and Risks More Generally (3/5)

- ▶ You should also consider **risks with patterns and tactics**
 - Are there any downsides or unintended consequences? Are there risks involved in technical design and implementation?
- ▶ Following Fairbanks (2010, p. 92), something like the following templates might work for rigor and consistency:
 - “ $\langle x \rangle$ is a priority due to a risk $\langle r \rangle$, and therefore we chose design $\langle y \rangle$ and accepted downside $\langle z \rangle$ ”
 - “ $\langle x \rangle$ is a priority due to a risk $\langle r \rangle$, and therefore we chose design $\langle y \rangle$ and accepted downside $\langle z \rangle$ leaving a residual risk $\langle \epsilon \rangle$ ”

Software Engineering and Risks More Generally (4/5)

- ▶ Overwhelmingly, the literature, whether from academia or industry, considers software engineering and software architecture risks through negative effects; an impact and its severity are taken as negative upon some assets
- ▶ However, this is a **simplified understanding** of risks
 - A risk can have **positive effects** too!
- ▶ Consider the following: *“our small startup with a funding for a project covering two years ends up being a unicorn”*
 - Regarding design, you would now probably think somewhat differently upon what to prioritize, including regarding QAs
 - For instance: maybe you only really need to scale to 100+ million users once you are a unicorn (and then you can rewrite the system outputted by your initial small project anyways)?

Software Engineering and Risks More Generally (5/5)

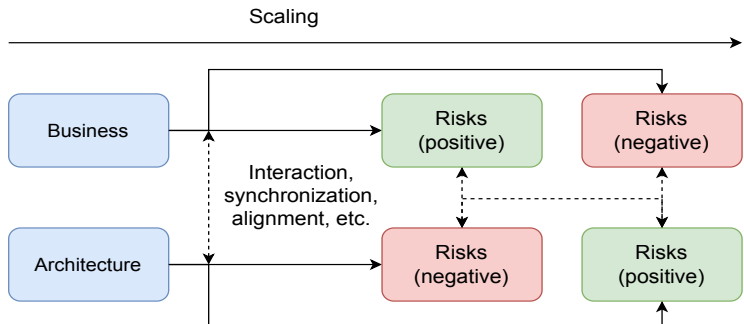


Figure 15: A Preliminary Sketch for Different Notions of Scaling and Risks

Software Architecture and Requirements Engineering (1/7)

- ▶ The distinction between detailed design and architecture is seen also in the artifacts produced in requirements engineering
- ▶ That said, there are some differences:

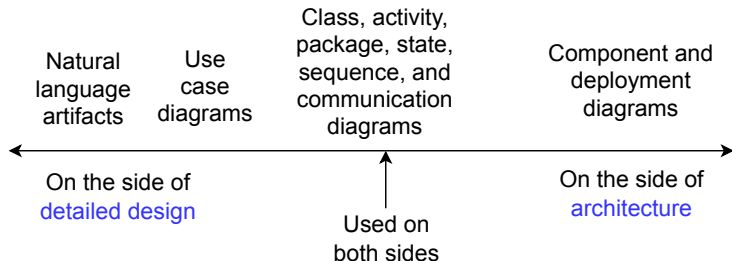


Figure 16: Req. Artifacts (derived from Nagl & Westfechtel 2024, p. 234)

Software Architecture and Requirements Engineering (2/7)

- ▶ A general problem is that requirement documents – no matter how rigorously elicited from stakeholders or otherwise written down, seldom contain explicit non-functional requirements
- ▶ Therefore, QAs must be typically derived or “**sniffed**” from the documents, to use the parlance of Bass et al. (2021)
- ▶ In fact, because most requirement documents catalog functional requirements, most of them do not even concern or affect an architecture being designed
- ▶ What was said earlier about prioritization is relevant because a perfect architecture would fulfill all QAs, and, therefore Bass et al. (2021) use a concept of **architecturally significant requirements** (ASRs)

Software Architecture and Requirements Engineering (3/7)

- ▶ For sniffing ASRs from functional requirements and their documents, there numerous things to look for, including:
 1. The perhaps easiest signal would be any explicitly **named technologies** or **named commercial vendors**
 2. How about other **external elements**; is integration required to other systems, whether software or hardware?
 3. Is anything said about **deployment**; will it run in a cloud, say?
 4. What do they say about **data**; how much, how fast, how often?
 5. What can you deduce about **networking**; will it rely on the standard TCP/IP stack; are WLAN/5G/Bluetooth/etc. involved; are there any custom network protocols?
 6. Does the intended **usage** say something about how many users there will be or whether internationalization is required?
 7. What does the domain say about **security** and **legal requirements**; do you need to comply with the GDPR, say?

Software Architecture and Requirements Engineering (4/7)

- ▶ Can you sniff ASRs from the following small imaginary snippet from an imaginary interview transcript with an executive?

“Another small company who provided and maintained our working hour allocation system went bankrupt and we are in a urgent need for a cheap new solution. Although there are only about three people occasionally using the system for about 200 employees, we operate in two different European countries. Our IT infrastructure is small. We have only four or five on-premise servers – if I remember correctly, and we encourage the bring your own device (BYOD) principle for our employees who like it too. However, we rely 100% on a SAP system for salary management based on the hours.”

Software Architecture and Requirements Engineering (5/7)

- ▶ Further conventional (qualitative) **requirements engineering methods** work also for ASRs/QAs; in-person or online interviews, focus groups, studying stakeholders' own documentation, observing people, surveys, and so forth
- ▶ For instance, the previous example mentioned on-premise servers; therefore, the company must have someone who understand about technology too and could thus presumably reason also about ASRs and QAs more broadly
- ▶ Bass et al. (2021) suggest also specific **ASR/QA workshops** in which stakeholders' own organizations are introduced, scenarios are brainstormed, evaluated, and prioritized, and finally refined and elaborated toward overall design goals

Software Architecture and Requirements Engineering (6/7)

- For documentation, something like the following might work:

Table 2: Example Tabulation (adopted from Bass et al. 2021)

QA	Refinement	ASR scenario
Performance	Throughput	The system must process 100 transactions per second at peak load.
Maintainability	Updates	Updates must be delivered automatically but with an opt-out function.
Maintainability	Modifiability	An employee must be able to add new tables to the database schemas within about two hours.
Usability	Learnability	A new student must be able to use the platform in about a hour at most.

Software Architecture and Requirements Engineering (7/7)

- ▶ The earlier tabulation is naïve for a real-world documentation, but each ASR scenario still satisfies (at least on paper) the earlier definition (on p. 10) for quality attributes
 - That is, the scenarios should be measurable and testable
- ▶ Fairbanks (2010, p. 250) goes further by recommending that each QA scenario should be **falsifiable**, meaning that they should be interpretable as dichotomous true/false tests
 - Although perhaps explicitly doable for QAs such as performance, QAs such as usability make that difficult; nevertheless, a falsifiable usability-QA is still better than a vague statement for developers to “*make it user-friendly*”

Quality Gates for ASRs/QAs (1/3)

- ▶ Provided that the ASRs/QAs elicited and defined are indeed measurable, testable, or even falsifiable, it may be possible to use **quality gates** (QGs) already during development
- ▶ The basic idea behind QGs is to provide some **checks and balances** especially for project owners or alike; they do not mean that a development should be halted upon failing to pass a gate (unless there is a catastrophic situation), but they rather provide tracking and signals about whether things are progressing as planned
- ▶ They are mostly applicable for large projects

Quality Gates for ASRs/QAs (2/3)

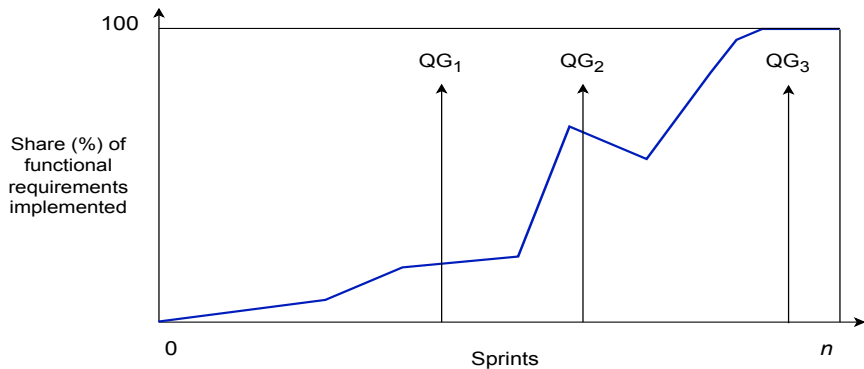


Figure 17: An Illustration of Quality Gates for Agile Development

Quality Gates for ASRs/QAs (3/3)

Table 3: An Illustration of Quality Gates for Falsifiable ASRs/QAs

QA	Refinement	QG ₁	QG ₂	QG ₃
Performance	Latency	–	✓	✓
Performance	Throughput	✓	✓	✓
Security	Confidentiality	–	–	✓
Security	Integrity	–	–	✓
Security	Availability	✓	–	–
Testability	...			
⋮				

Reference Architectures and Requirements (1/4)

- ▶ As the name indicates, a software reference architecture provides a reference, a “*guardrail architecture*” or a “*blueprint architecture*” to follow in “actual” architectures
- ▶ They are useful for promoting reuse of **best practices**, for facilitating interoperability, and for conveying the intended meaning in standards and regulations, among other things
 - Thus, they are useful also for understanding the requirements in a particular domain or of a technology
- ▶ For instance, CPython is the reference architecture and implementation for an interpreter of the Python programming language, although it is also the *de facto* implementation because solid alternatives have not emerged thus far

Reference Architectures and Requirements (2/4)

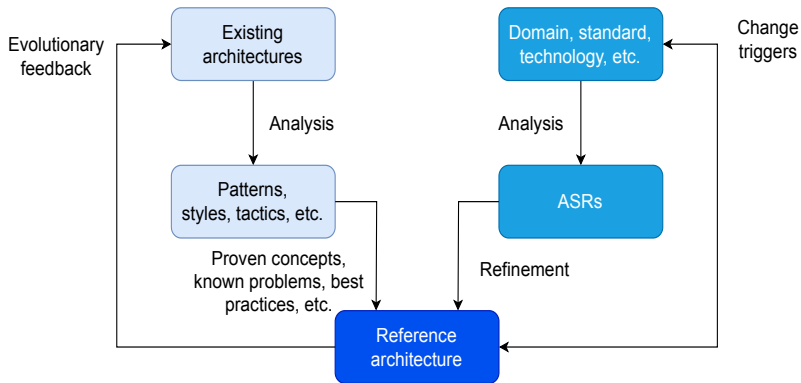


Figure 18: A Design Process for a New Reference Architecture (adopted and modified from Muller & Hole 2007, p. 11)

Reference Architectures and Requirements (3/4)

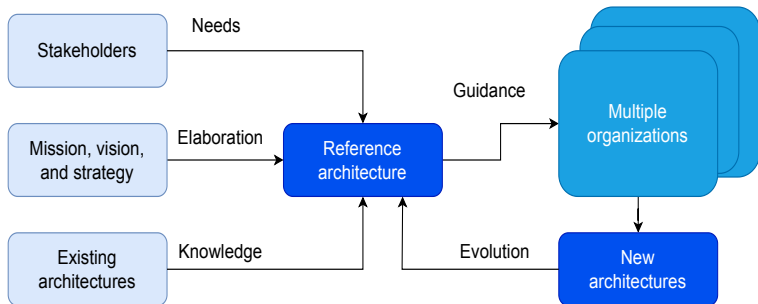


Figure 19: Another Design Process for a New Reference Architecture (adopted and modified from Muller & Hole 2007, p. 15)

Reference Architectures and Requirements (4/4)

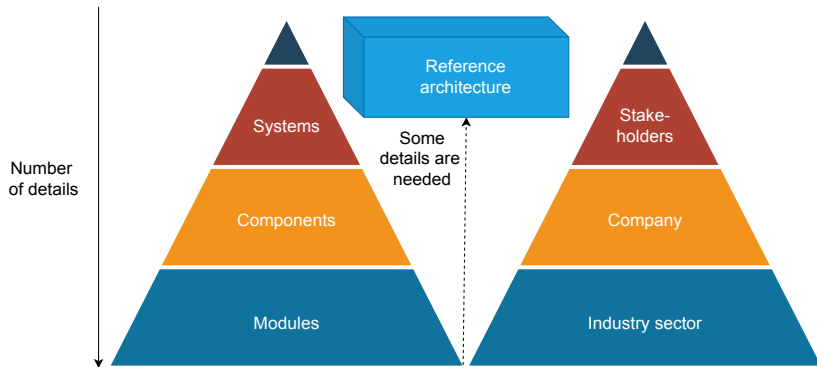


Figure 20: A Further Design Process for a New Reference Architecture (adopted and modified from Muller 2025, p. 14)

Question

What Might Be a Risk
with Reference Architectures?

Standards and Requirements (1/3)

- ▶ Implicitly or explicitly, standards, including for protocols, are often dealing with reference architectures at least in theory
- ▶ If working with **standardization** in itself, it is necessary to know a particular domain as well as the general design principles applicable to a particular standardization body
- ▶ Although reference architectures evolve, those bound to standards almost always have to ensure long evolution cycles and usually **backwards compatibility** too
 - If an ASR is explicitly embedded to a standard, it may have far-reaching consequences for decades (or more) to come!

Question

What Might Be One of the Internet's Core ASR?

Standards and Requirements (2/3)

- ▶ IETF (1996) has noted the general architectural design issues affecting the Internet:

1. “**Heterogeneity** *is inevitable and must be supported*”
 - Heterogeneity is about the compatibility QA category
2. “*If there are several ways of doing the same thing, **choose one***” (thus, even if you do twin peaks modeling, one architecture must eventually be chosen for an implementation)
3. “*All designs must **scale** readily to very many nodes per site and to many millions of sites*” (thus, scalability is a QA too)
4. “**Performance and cost** *must be considered as well as functionality*” (thus, two general QAs & one constraint)
5. “*Keep it **simple**. When in doubt during design, choose the simplest solution*” (thus, avoiding complexity is another QA)

Standards and Requirements (3/3)

6. “**Modularity** is good. If you can keep things separate, do so.”
7. “it is better to **adopt** an almost complete solution now, rather than to wait until a perfect solution can be found.”
8. “**Avoid options** and parameters whenever possible. Any options and parameters should be configured or negotiated dynamically rather than manually.”
9. “Be **strict when sending and tolerant when receiving**. Implementations must follow specifications precisely when sending to the network, and tolerate faulty input from the network. When in doubt, discard faulty input silently”
10. “Be **parsimonious with unsolicited packets**, especially multicasts and broadcasts”

Domain Modeling and Requirements (1/6)

- ▶ Yet, a lot of software is designed and developed without any explicit or particular “stakeholders” – consumer markets, commercial-off-the-shelf (COTS) software, and so forth
- ▶ Modeling helps at defining ASRs/QAs in these many cases
- ▶ By modeling, we do **not** mean model-driven engineering strictly speaking; hence, recall what was said in the first lecture about diagrams, syntax, and semantics
- ▶ By modeling, in the present context, we most of all mean gaining better understanding about a given domain within which explicit stakeholders may not always exist

Domain Modeling and Requirements (2/6)

- ▶ Even with the previous reservations, one frequently hears various objections to modeling, including from students
- ▶ Fairbanks (2010, p. 218) lists the following five:
 1. *"You already know your domain"*
 2. *"The domain is too simple to bother modeling it"*
 3. *"The domain is irrelevant to your architecture choices"*
 4. *"It is someone else's job to do requirements"*
 5. *"The best way to learn the domain is" to "write code"*
 6. *"Domain modeling is an open-ended analysis paralysis activity"*

Domain Modeling and Requirements (3/6)

► Fairbanks (2010, p. 130) answers:

1. *“Undoubtedly this is often true, though it is less often true in ‘boring’ business domains than in ‘sexy’ internet and systems domains, where developers enjoy learning about the domain”*
2. *“Domains are rarely much simpler than”* what you think
3. *“While the domain influences your architecture, it is a mistake to believe that your architecture is latent in the domain, waiting for you to discover it”*
4. *“Learning the domain by writing code is indeed a good way to learn it, but there are cases when this is impossible or impractical”*, and it is oftentimes *“overwhelmingly cost-effective to model on paper first”*
5. *“This is a real danger”* and therefore you *“want to model just enough of the domain”*

Domain Modeling and Requirements (4/6)

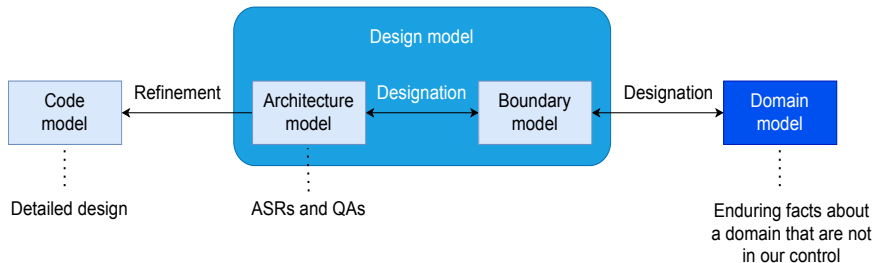


Figure 21: A Blueprint for Domain Modeling

Domain Modeling and Requirements (5/6)

- ▶ By recalling the concepts from the first lecture, with domain modeling we are thus at the intersection of a domain and a design, including an architectural design
 - By designation, it is meant that we seek a correspondence between a high-level domain model and a custom boundary model, and a correspondence between the custom boundary model and the architecture model we are designing
- ▶ Fairbanks (2010) suggests to construct the domain model by providing different **views** for it, including with domain-specific concepts, snapshots, statistics, scenarios, and alike
 - The same applies for the design model

Domain Modeling and Requirements (6/6)

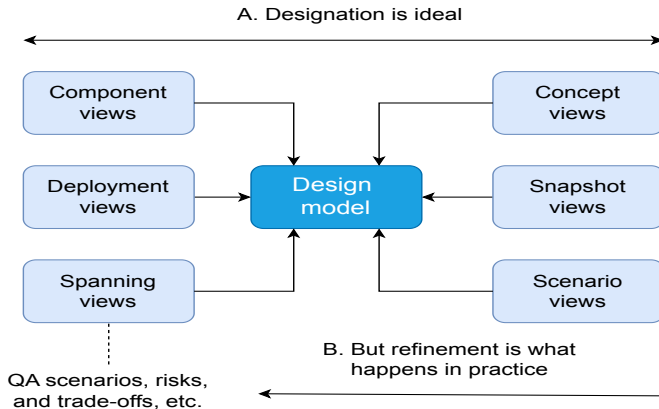


Figure 22: Another Blueprint for Domain Modeling

Software Architecture and Non-Technical QAs (1/3)

- ▶ A further takeaway is that domain modeling is also about non-technical QAs for software architectures, among them **marketability** and related business factors
 - If you look at what Azure or AWS offer for instance, there are a lot of reference architectures and promises of technical QAs for cloud-native applications; hence, for them, ready-made blueprint architectures are also about business
- ▶ Sometimes business factors (such as time-to-market, say) may outweigh technical quality attributes
 - Also business factors should be preferably measurable

Software Architecture and Non-Technical QAs (2/3)

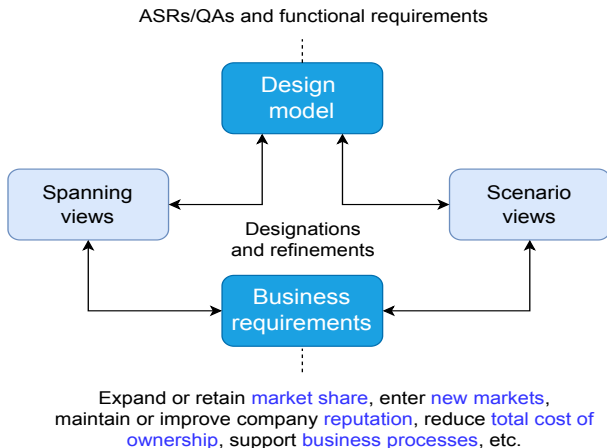


Figure 23: A Further Blueprint for Domain Modeling

Software Architecture and Non-Technical QAs (3/3)

- ▶ Thus, designation is ideal also in a sense that concepts such as **total cost of ownership** are directly related also to business requirements, including everything from labor costs to hardware purchases and various indirect costs
 - Azure and AWS are not that cheap, but if scalability is a business requirement, they may still outweigh competitors
- ▶ While people often think about concepts such as **technical debt** ($\implies$ maintainability QA) in engineering terms, many of the concepts have their business equivalents

Deployment Modeling and Requirements (1/8)

- ▶ Recall the important point that many QAs require understanding a system's **runtime** behavior; therefore, “deployment modeling” (for a lack of a better term) is important too for guaranteeing some of the ASRs derived
- ▶ As was noted (on p. 37), deployment diagrams are on the side of architecture and ASRs rather than “traditional requirements engineering”, and it is thus useful to briefly illustrate them
- ▶ Expectedly, UML provides its own notation for deployment diagrams; it speaks about nodes and such (see Section 19 in OMG 2017) but in what follows the terminology from the first lecture is used and liberty is taken for drawing

Deployment Modeling and Requirements (2/8)

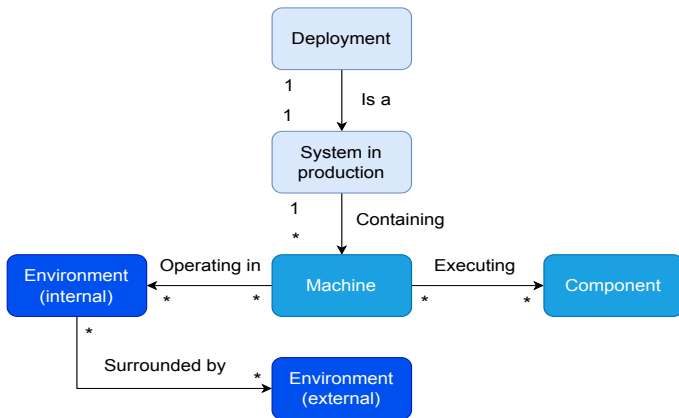


Figure 24: Further Elaboration of the Basic Concepts

Deployment Modeling and Requirements (3/8)

- ▶ Assuming that we know the internals of our components thoroughly, already the previous simple figure allows asking many relevant questions for meeting some notable QAs
 - **What** are the machines in which our components execute?
 - **How capable** the machines are?
 - **How many** machines there are in our system?
 - **Where** do the machines operate?
- ▶ Answers to these questions depend wildly between domains
 - The answers for a mobile application are entirely different than the answers for a large cloud-native machine learning application, for instance

Deployment Modeling and Requirements (4/8)

- ▶ During the later lectures – particularly when addressing scalability and performance, deployment diagrams are discussed in more detail
- ▶ For the present purposes, it suffices to discuss a little about the components we are executing in machines deployed in some environment from a **maintenance**-QA perspective
- ▶ We are executing something in production, and, hence, **what should we model and catalog, and how?**
 - Or maybe “*you already know your domain*” (cf. p. 57)?

Deployment Modeling and Requirements (5/8)

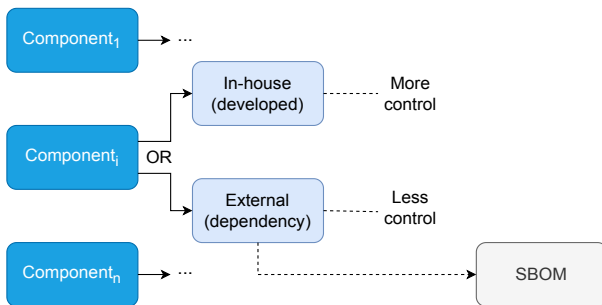


Figure 25: Further Elaboration of the Basic Concepts

Deployment Modeling and Requirements (6/8)

- ▶ A software bills of materials (SBOM) is a systematic catalog of “primary” (typically top-level) external dependencies
 - Importantly, SBOMs are mandated by recent laws both in the European Union and in the United States; therefore, **having a SBOM is a mandatory ASR** (in Europe by 2027)
- ▶ For the present purpose and given the focus on maintenance, we should obviously know what our systems really contain
 - The larger a system, the more relevant a SBOM is
- ▶ There are many SBOM formats and tools for these; the one from OWASP (2025) suffices for the present purposes because its schema is extensive – if not exhaustive

Deployment Modeling and Requirements (7/8)

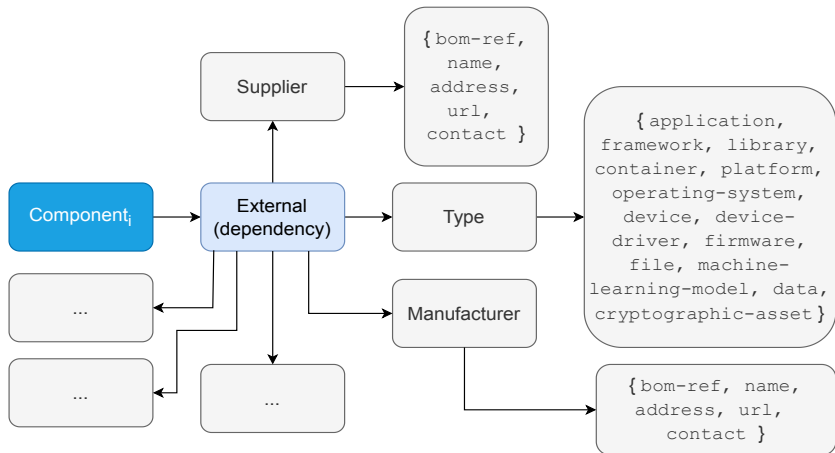


Figure 26: Part of the CycloneDX SBOM's Schema (OWASP 2025)

Deployment Modeling and Requirements (8/8)

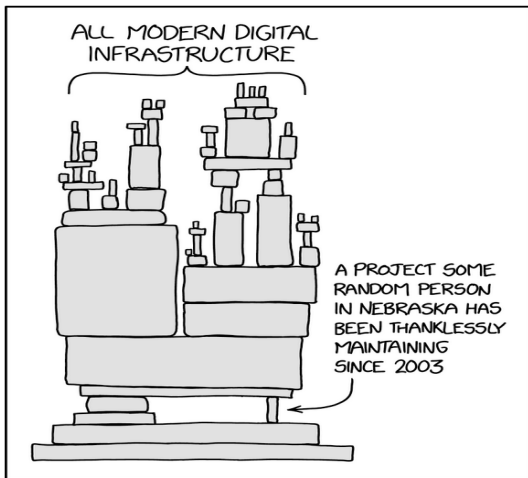


Figure 27: <https://xkcd.com/2347/>

Questions?

Case #2

- ▶ Pick your favorite LLM (or if you do not have access use SDU's Copilot instance; <https://copilot.sdu.dk>)
- ▶ Use it to “elicit” QAs; prompt it to prioritize them and to provide brief justifications and rationales for them
- ▶ You can try whether templates (such as on p. 34) might help
- ▶ The case is:

“We are going to install about 500 sensors to our two large premises for helping to optimize electricity costs by shutting lightning in rooms whenever there is sufficient daylight. The sensors will be provided by three different vendors. We are a research institution and want to use the sensors also to collect continuous streams of big data that can be used in research for studying automated energy efficiency of buildings.”

Thank you

Questions?

Additional References

1. Yang, C. & Liang, P. & Avgeriou, P. (2016): A Systematic Mapping Study on the Combination of Software Architecture and Agile Development. Journal of Systems and Software 111, pp. 157–184,
<https://doi.org/10.1016/j.jss.2015.09.028>,
2. Muller, G. & Hole, E. (2007): Reference Architectures; Why, What and How. White Paper Resulting from Architecture Forum Meeting, Stevens Institute of Technology, Embedded Systems Institute, https://www.architectingforum.org/whitepapers/SAF_WhitePaper_2007_4.pdf
3. Muller, G. (2025): A Reference Architecture Primer. A presentation for the Gaudi project, University of South-Eastern Norway,
<https://gaudisite.nl/ReferenceArchitecturePrimerSlides.pdf>
4. IETF (1996): Architectural Principles of the Internet. RFC 1958,
<https://www.ietf.org/rfc/rfc1958.txt>
5. OMG (2017): OMG[®] Unified Modeling Language[®] (OMG UML[®]), Version 2.5.1, formal/2017-12-05, <https://www.omg.org/spec/UML/2.5.1/PDF>
6. OWASP (2025): CycloneDX v1.7 JSON Reference,
<https://cyclonedx.org/docs/1.7/json/>